

Lecture 10 Sorting(III)

Stable Sort Algorithms

A stable sort is the sort that the **relative orders** of **equal elements** are kept.

An unstable sort **may or may not** keep the relative orders of equal elements.

It may matter sometimes. Such as sorting students by test scores. We should keep the order of name. So we need stable sort.

Stability of Merging

If we merge two same elements, we take the **left element** to keep the merge sort **stable**.

Summary: Comparison based Sorting Methods

Sorting Method	Time Complexity (Worst Case)	Time Complexity (Average Case)	Stability	Space Complexity
Insertion sort	$O(n^2)$	$O(n^2)$	Y	$O(1)$
Shell sort			N	$O(1)$
Bubble sort	$O(n^2)$	$O(n^2)$	Y	$O(1)$
Heap sort	$O(n \log n)$	$O(n \log n)$	N	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	Y	$O(n)$
Quick sort	$O(n^2)$	$O(n \log n)$	N	$O(\log n)$

In general, swapping two adjacent elements is stable. Swapping two elements with large distance is unstable.

Best choice in practice is **quick sort**.

It can be proved that any **comparison-based sorting algorithm** will need to carry out at least **$O(n \log n)$ operations on average and worst case**.

The theorem can be proved by **decision tree** of insertion. For an array with n elements, there are $n!$ situations totally. For each situation, it is a leaf node of the decision tree. A tree with k layers has at most 2^k leaf node. So $2^k \geq N!$. Thus $k \geq \log(N!)$. It can be simplified by Stirling's Approximation such that $k \geq \Omega(n \log n)$.

Non-Comparison Sorts

Bucket Sort

Premise:

The values in the list to be sorted can repeat but the values **range in a limit**.

Idea:

The values are in the range $0 \dots m - 1$, start with m **empty buckets** numbered 0 to $m - 1$, scan the list and **place element i in the i -th bucket** and then **output the bucket in order**.

- If we are sorting values:
Just need **counter** for each bucket and increases it rather than input data in the bucket.
- If we are sorting entries according to keys
We can use **queue** to keep stable of the sort.

The time complexity is $O(m + n)$.

If $n \gg m$, the time complexity approximately is $O(n)$. But when $m \gg n$, its time complexity will become much larger.

Radix Sort (基数排序)

- Digit-by-digit sort
 - Every integer can be represented by at most k digits.
 - $d_0 d_1 \dots d_{k-1}$ where d_i are digits in base r .
 - d_0 is the **Most Significant Digit(MSD)**
 - d_{k-1} is the **Least Significant Digit(LSD)**.
- For each d , it satisfies that $0 \leq d \leq r - 1$.

The most common method is **LSD radix sort**.

- Ensure the number of digits.
- Handle the digits from LSD **iteratively**
 - Allocate the elements into **corresponding buckets** according to **current digit**.
 - Fetch the elements in buckets **sequentially** and **merge into a new array**.
- Terminal condition: Handle MSD.

Suppose k digits and n numbers, then the time complexity is $O(k(n + r))$.

Increasing the base r decreases the number of passes k .

Similarly, the "integer" can be entries with many keys. We can process the radix sort from the least significant key to the entries. And the list of entries is **lexically sorted**.

For **MSD radix sort**:

- Ensure the number of digits.
- Handle the digits from MSD **recursively**
 - Allocate the elements into corresponding buckets according to current digit.
 - **Recursively** handle the elements in each **non-empty bucket**.
 - Terminate when **only one element in buckets** or **handle LSD**
 - **Return the elements into original bucket** by fetching **current buckets sequentially**.

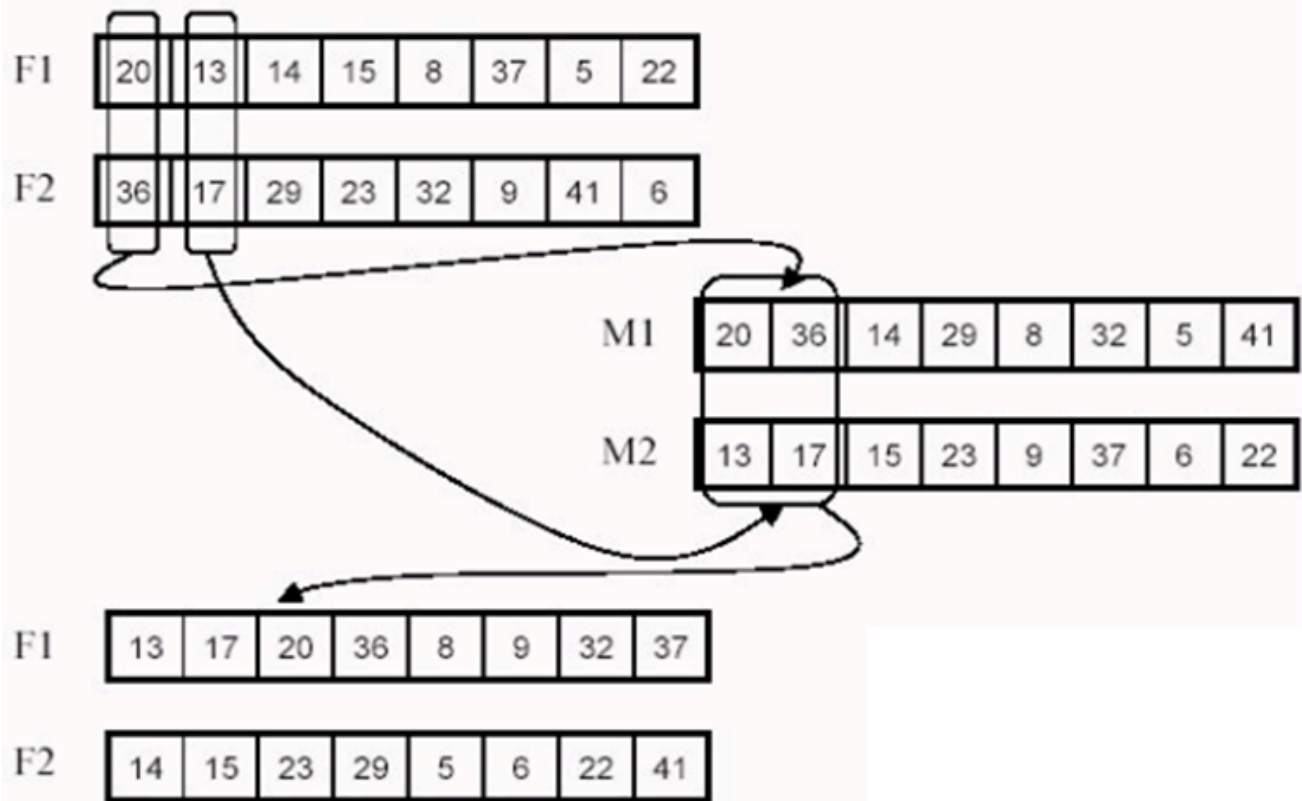
External Sort

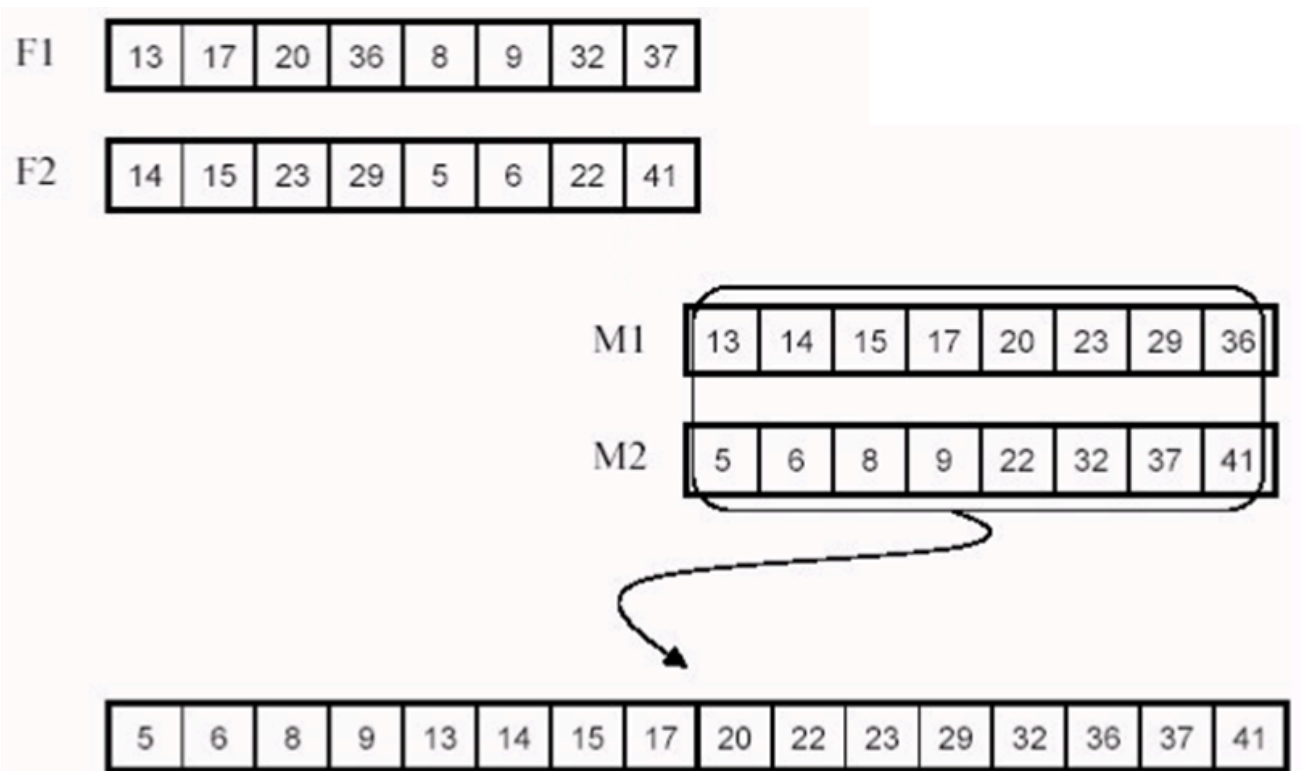
The principle goal of external sorting:

Minimize the times of disk accessing operations.

Simple Approach of External Sort

- **Partition** the file into two files, $F1$ and $F2$.
- **Take the first record** from each file and **write them in sorted order to a new file**, say $M1$.
- **Repeat** the process with the record from each file **but write to a second file** $M2$.
- Repeat until $F1$ and $F2$ have been done. $M1$ and $M2$ consist of ordered pairs of records.
- Repeat step 2-4 but **merge pairs of records instead of single records**, producing sorted runs of length 4.
- Continue and building longer and longer runs.





Reducing the Number of Passes

For a file with n records, simple merge sort requires $\log_2 n$ passes to read or write each record.

Two approaches to reduce the number of passes:

- Break the file into **large initial runs**.
- Merge the runs by **multi-way**.